

Assertion Messages and Good Practices

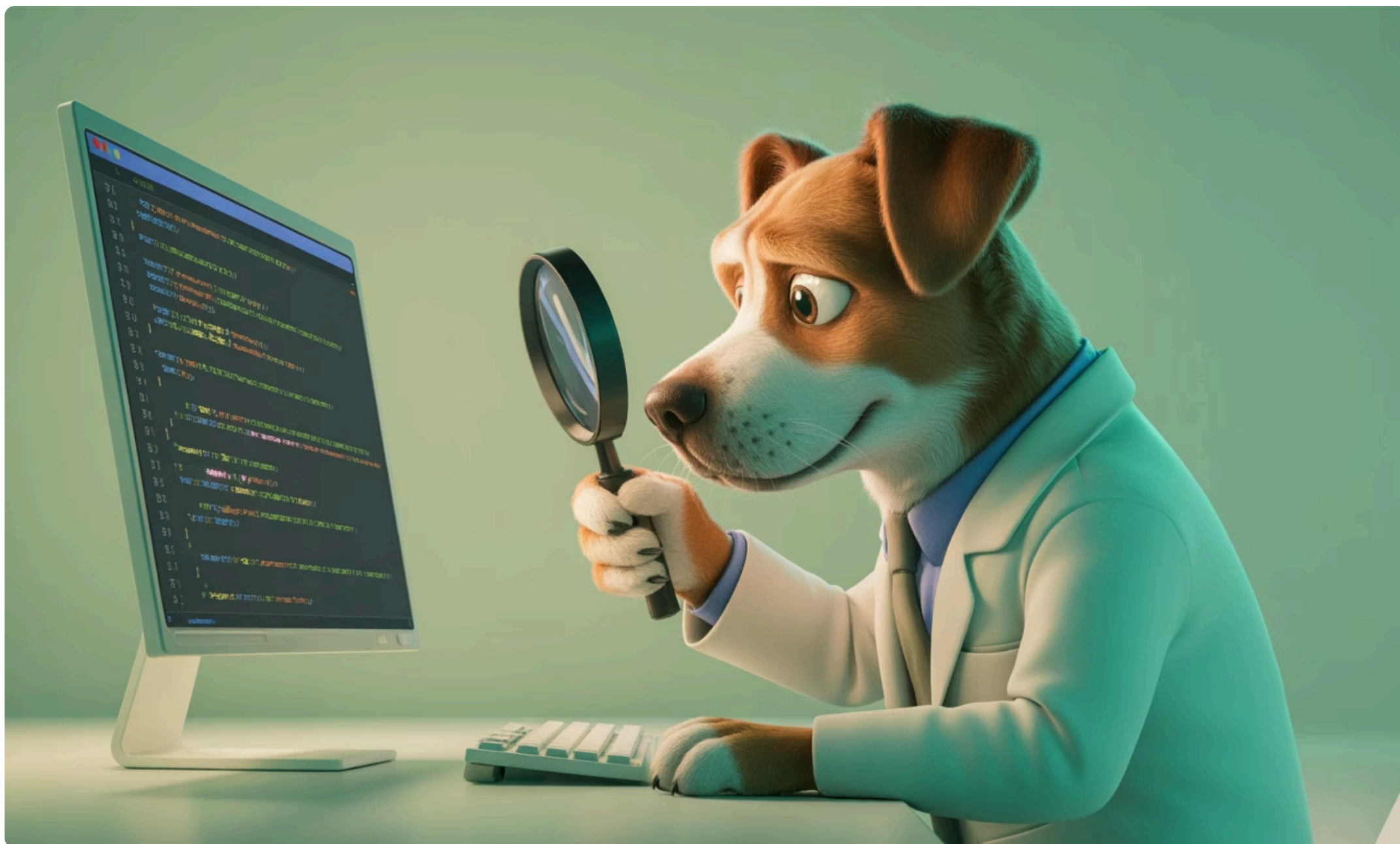


Understanding how to write clear and efficient test assertions in Java unit testing

 by Rodrigo Martins Pagliares

Last update: 11/25/2025

Assertion Messages and Good Practices



Optional Message Parameter

`assertEquals` supports an optional message parameter to clarify failures

Descriptive Messages

Adding a descriptive message helps other developers understand why a test failed

Include Context

Messages may include input parameters and expected/actual results

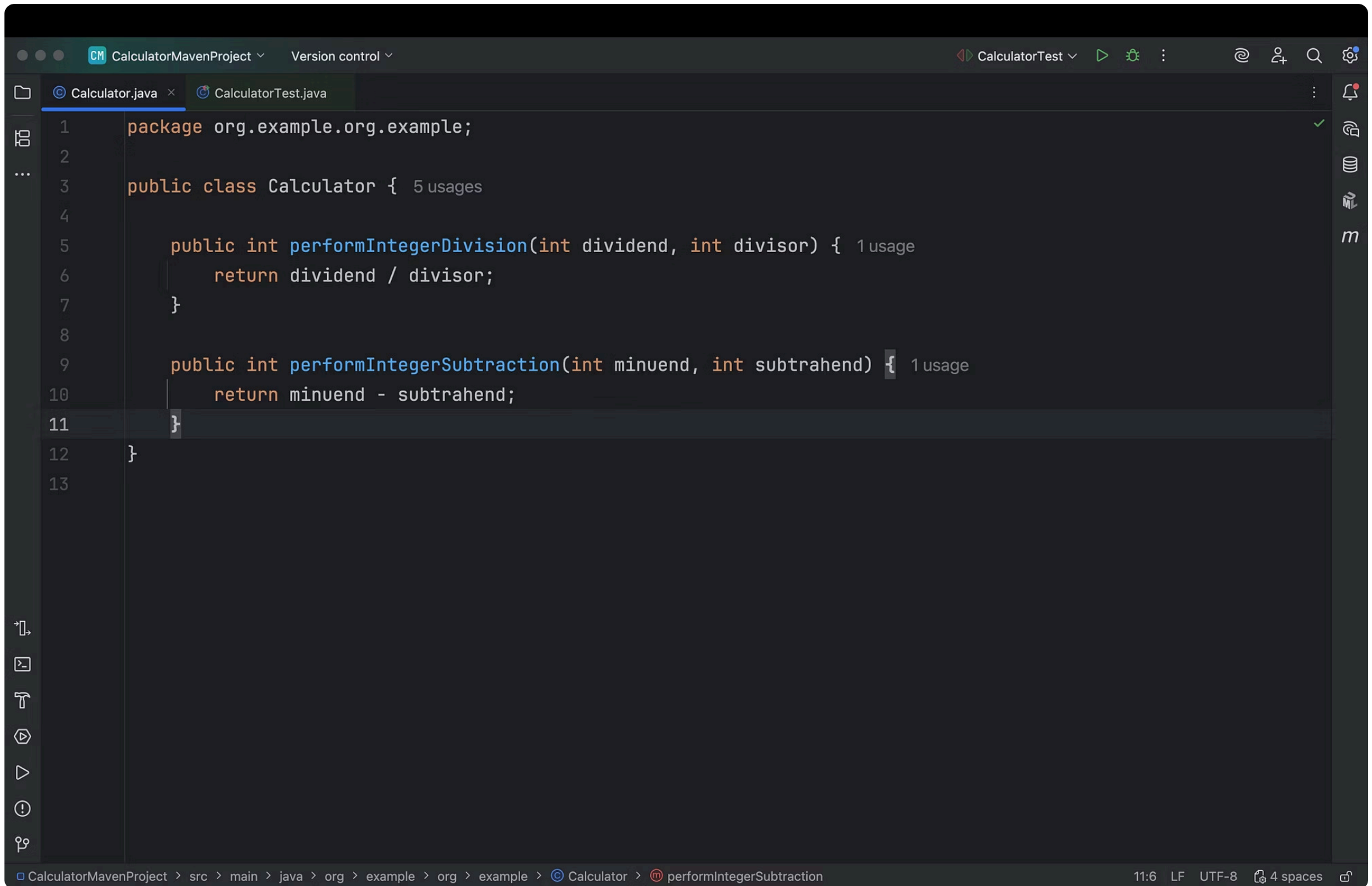
Faster Debugging

Clear failure messages improve collaboration and debugging speed

Best Practice

Although optional, using messages is widely considered a good practice

The method performIntegerSubtraction



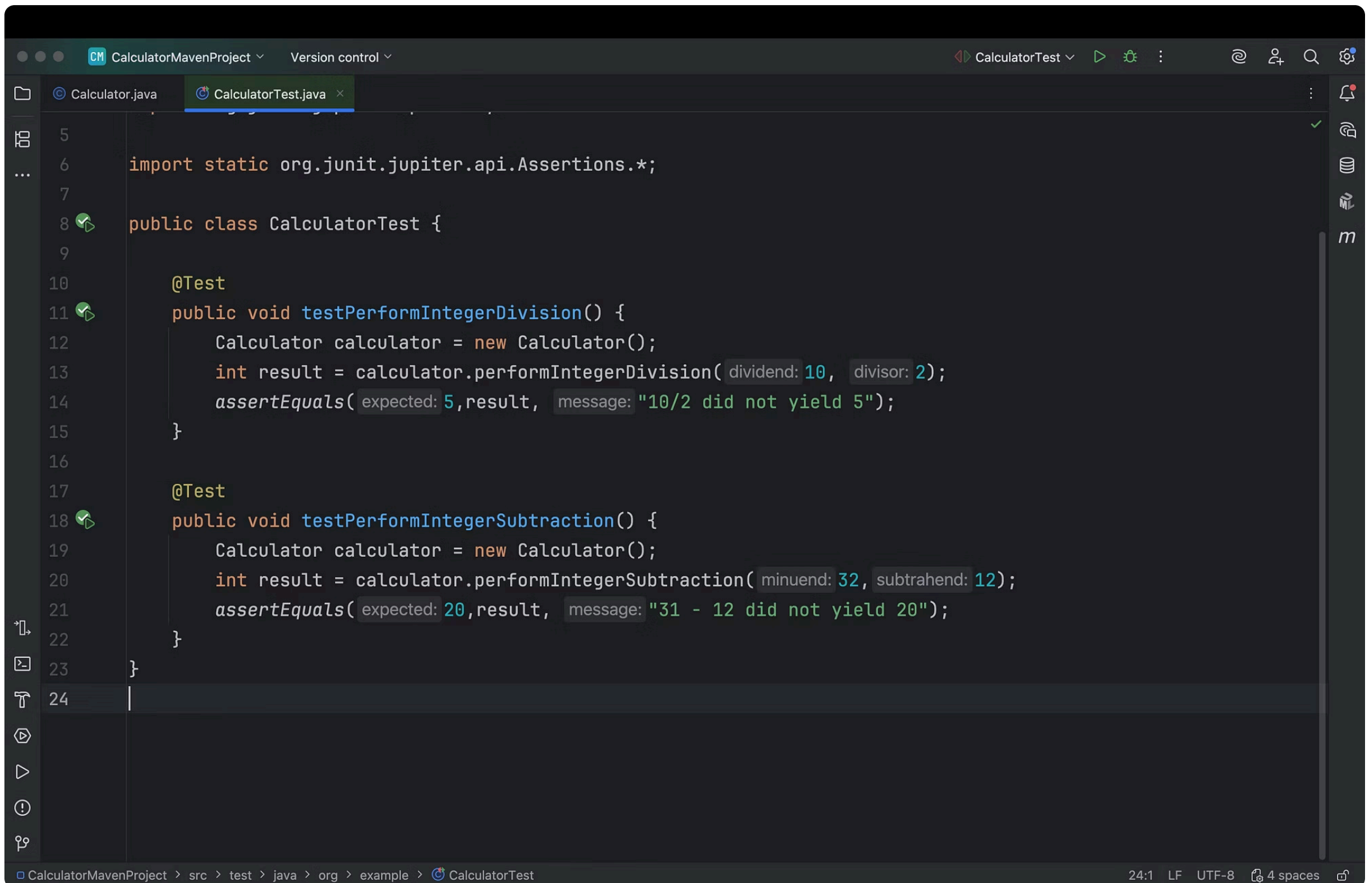
The screenshot shows an IDE window for a project named "CalculatorMavenProject". The "Calculator.java" file is open, displaying the following code:

```
1 package org.example.org.example;
2
3 public class Calculator { 5 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend / divisor;
7     }
8
9     public int performIntegerSubtraction(int minuend, int subtrahend) { 1 usage
10        return minuend - subtrahend;
11    }
12 }
13
```

The "performIntegerSubtraction" method is highlighted with a dark background. The IDE interface includes a top bar with project and version control information, a left sidebar with file explorer and tool icons, and a bottom status bar showing the current file path and encoding details.

The unit test for testing the method performIntegerSubtraction

The testPerformIntegerSubtraction unit test method

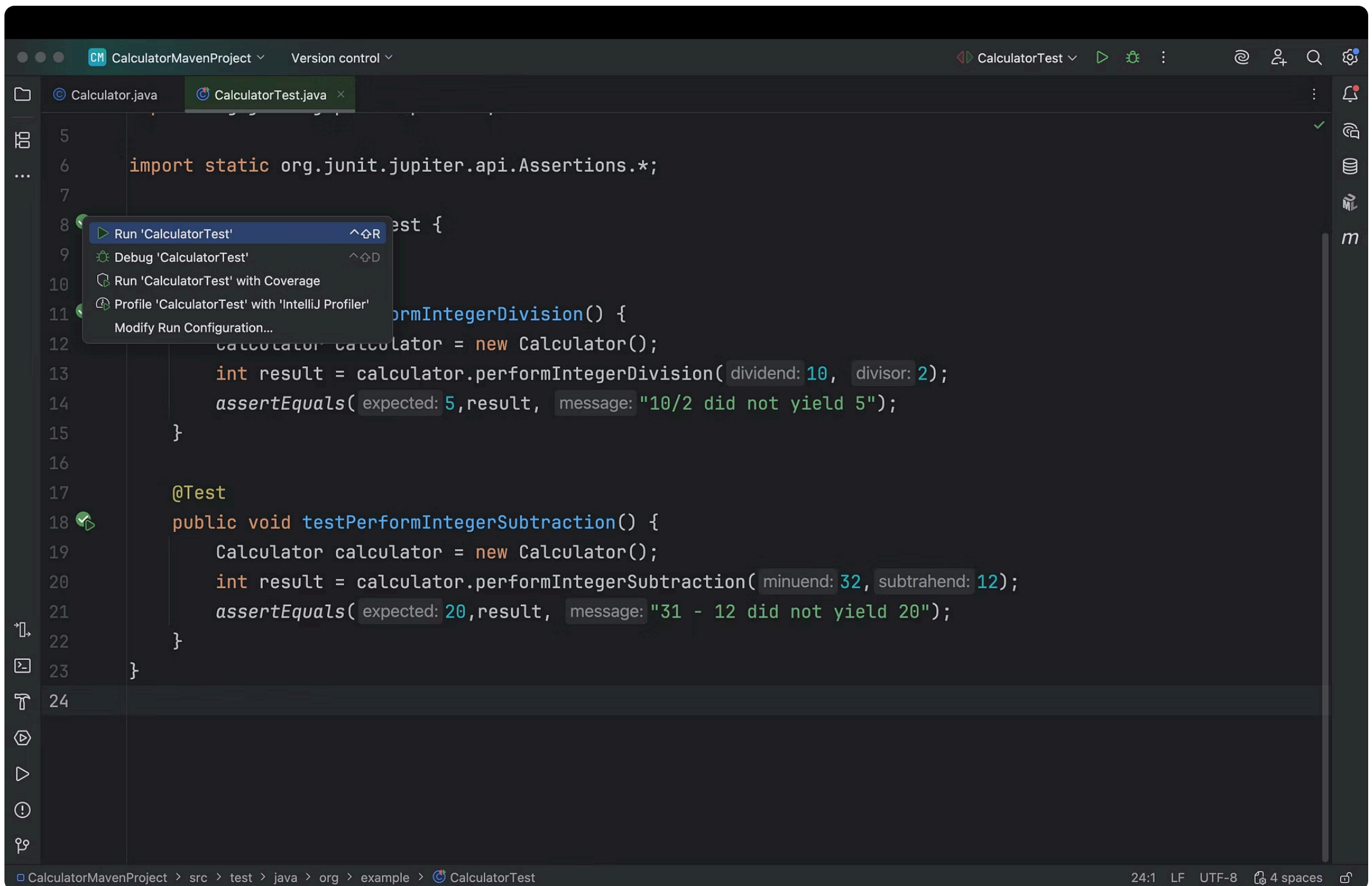


```
5
6 import static org.junit.jupiter.api.Assertions.*;
7
8 public class CalculatorTest {
9
10     @Test
11     public void testPerformIntegerDivision() {
12         Calculator calculator = new Calculator();
13         int result = calculator.performIntegerDivision( dividend: 10, divisor: 2);
14         assertEquals( expected: 5, result, message: "10/2 did not yield 5");
15     }
16
17     @Test
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int result = calculator.performIntegerSubtraction( minuend: 32, subtrahend: 12);
21         assertEquals( expected: 20, result, message: "31 - 12 did not yield 20");
22     }
23 }
24
```

CalculatorMavenProject > src > test > java > org > example > CalculatorTest

24:1 LF UTF-8 4 spaces

Running the unit test for testing the method `performIntegerSubtraction`



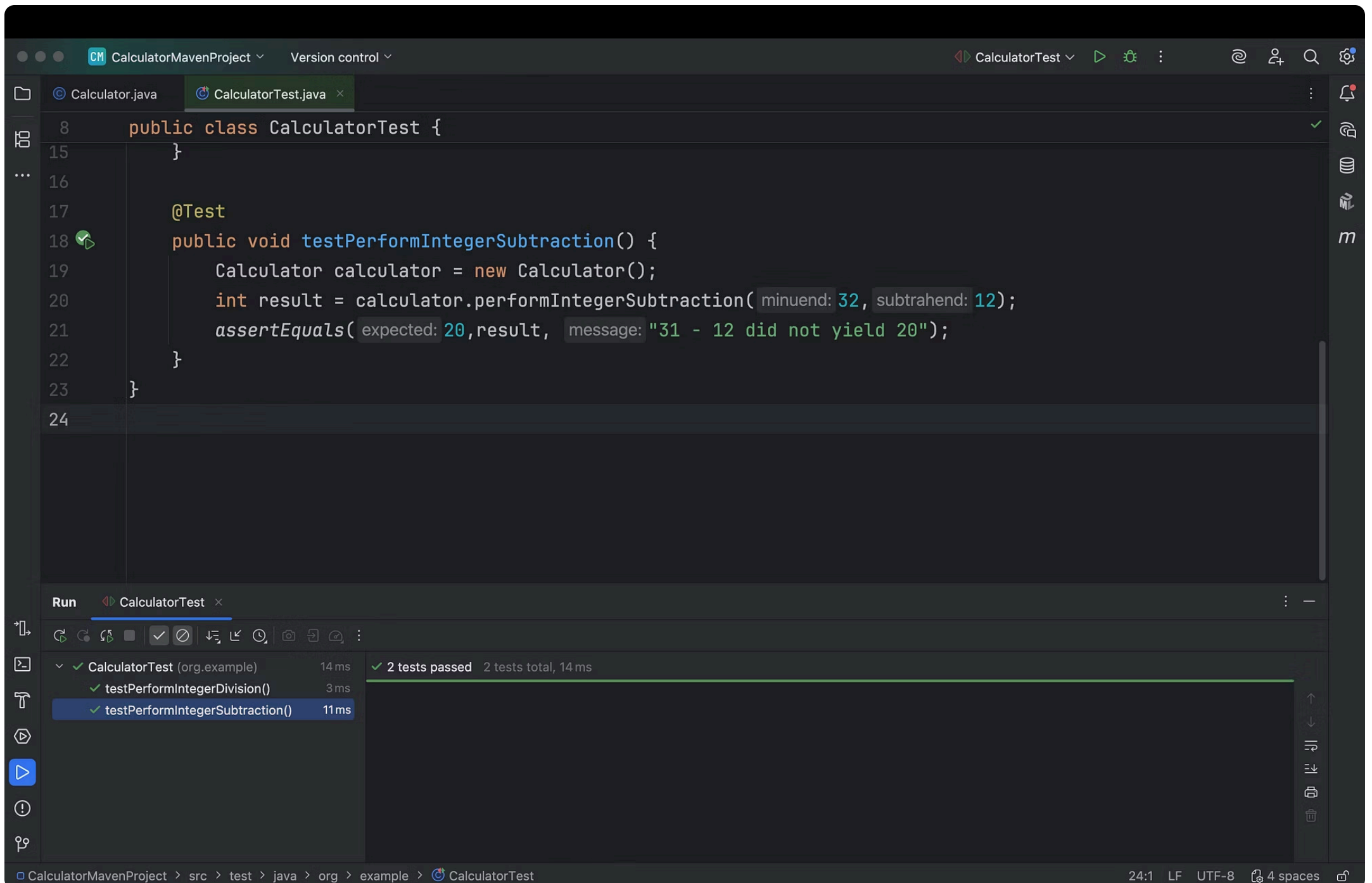
The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The file 'CalculatorTest.java' is open, displaying a unit test for the 'performIntegerSubtraction' method. A context menu is open over the test method, showing options to run, debug, or profile the test. The test method is annotated with '@Test' and contains the following code:

```
import static org.junit.jupiter.api.Assertions.*;

@Test
public void testPerformIntegerSubtraction() {
    Calculator calculator = new Calculator();
    int result = calculator.performIntegerSubtraction( minuend: 32, subtrahend: 12);
    assertEquals( expected: 20, result, message: "31 - 12 did not yield 20");
}
```

The IDE interface includes a top toolbar with icons for running, debugging, and other actions. The bottom status bar shows the current file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest'.

Running the unit test for testing the method `performIntegerSubtraction` (contd.)



The screenshot shows an IDE window for a project named "CalculatorMavenProject". The active file is "CalculatorTest.java". The code defines a class "CalculatorTest" with a single test method "testPerformIntegerSubtraction()". The test method creates a "Calculator" object, calls "performIntegerSubtraction(32, 12)", and asserts that the result is 20. The test message is "31 - 12 did not yield 20".

```
8 public class CalculatorTest {
15 }
16
17 @Test
18 public void testPerformIntegerSubtraction() {
19     Calculator calculator = new Calculator();
20     int result = calculator.performIntegerSubtraction( minuend: 32, subtrahend: 12);
21     assertEquals( expected: 20,result, message: "31 - 12 did not yield 20");
22 }
23 }
24
```

The "Run" tab at the bottom shows the test results for "CalculatorTest (org.example)". The test "testPerformIntegerSubtraction()" passed in 11ms. The overall test result is "2 tests passed" (2 tests total, 14 ms).

CalculatorMavenProject > src > test > java > org > example > CalculatorTest 24:1 LF UTF-8 4 spaces

Refactoring the unit test

The screenshot shows an IDE window for a project named "CalculatorMavenProject". The "CalculatorTest.java" file is open, displaying the following code:

```
8 public class CalculatorTest {
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int minuend = 32;
21         int subtrahend = 12;
22         int expectedResult = 20;
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, message: minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28 }
29
```

Below the code editor, the "Run" tab is active, showing the execution results for "CalculatorTest". The output indicates that 2 tests passed (2 tests total, 17 ms):

- ✓ testPerformIntegerDivision() 5 ms
- ✓ testPerformIntegerSubtraction() 12 ms

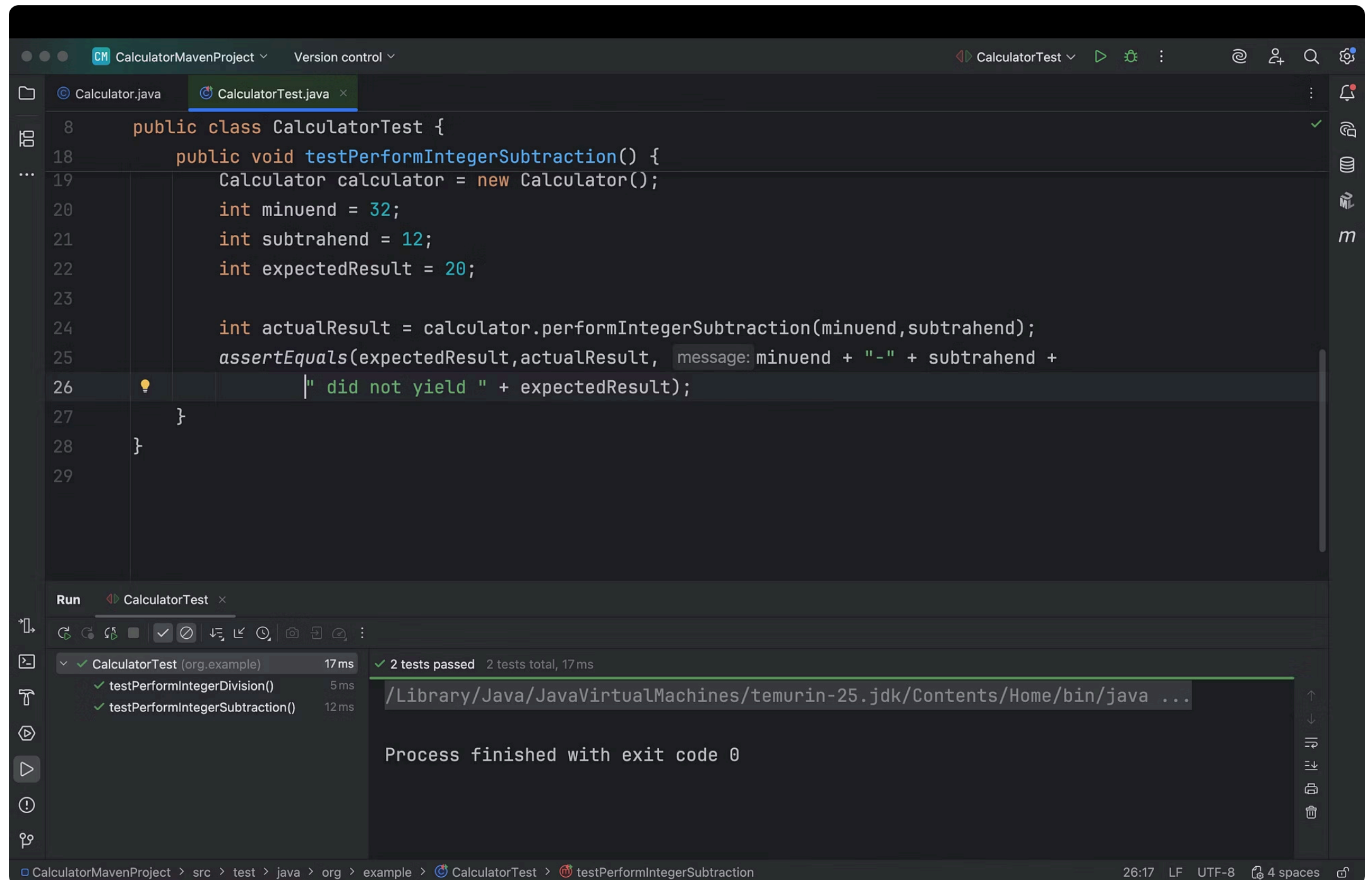
The console output shows the command path: `/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...` and the message: `Process finished with exit code 0`.

The bottom status bar shows the file path: `CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction`, along with the time `26:17`, line `LF`, encoding `UTF-8`, and `4 spaces`.

Problem with this Refactoring

Lower performance since error message is computed every time you run the test.

It can make difference if you have too many unit tests.



The screenshot shows an IDE window with two tabs: `Calculator.java` and `CalculatorTest.java`. The `CalculatorTest.java` file contains the following code:

```
8 public class CalculatorTest {
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int minuend = 32;
21         int subtrahend = 12;
22         int expectedResult = 20;
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, message: minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28 }
29
```

Below the code editor, the `Run` tab is active, showing the execution results for `CalculatorTest`. The results indicate that 2 tests passed (2 tests total, 17 ms). The tests are:

- ✓ `testPerformIntegerDivision()` (5 ms)
- ✓ `testPerformIntegerSubtraction()` (12 ms)

The output window shows the command used to run the tests:

```
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...
```

And the final message:

```
Process finished with exit code 0
```

The bottom status bar shows the project path: `CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction`, along with the time (26:17), line (LF), and encoding (UTF-8).

```
assertEquals(expectedResult, actualResult, minuend + "-" + subtrahend +  
    " did not yield " + expectedResult);
```

Performance Concerns and Lambda Messages



01

String Computation Cost

When **messages** are **plain Strings**, they are **computed every time** the test runs

02

Performance Impact

In **large suites**, **repeated computation** can slightly **slow down** test **execution**

03

Lambda Solution

Using a lambda `() -> "error message"` **delays message creation**

04

Conditional Execution

Lambdas compute the message **only if the assertion fails**

05

Optimized Approach

This **approach optimizes performance** without sacrificing clarity

Solution to the Refactoring: Lambda functions

Transform the third argument (error message) into a Lambda Function (Java SE 8).

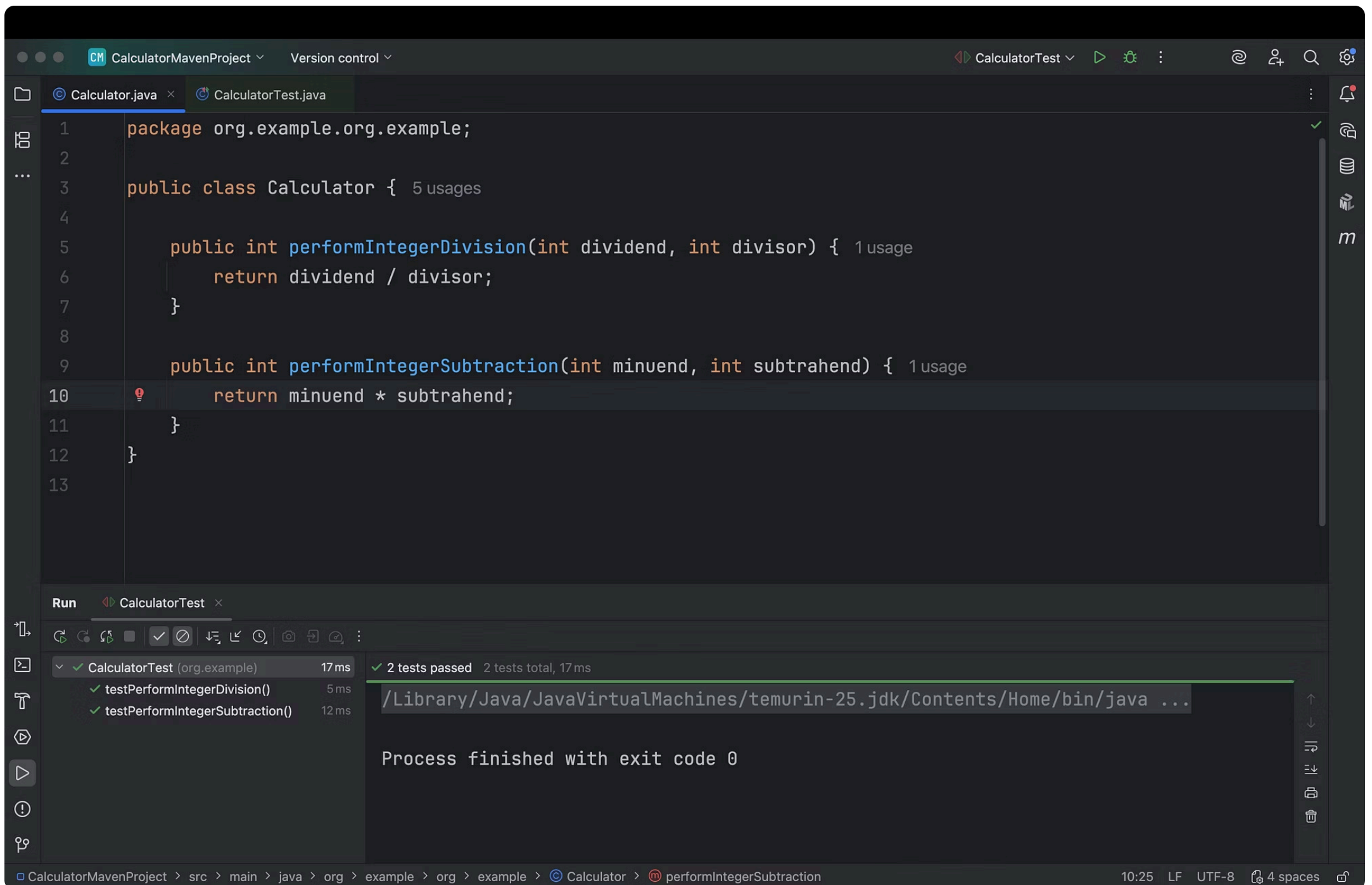
Only executed if test fails!

The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The 'CalculatorTest.java' file is open, displaying the following code:

```
8 public class CalculatorTest {
17     @Test
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int minuend = 32;
21         int subtrahend = 12;
22         int expectedResult = 20;
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28 }
29
```

Below the code editor, the 'Run' tab is active, showing the execution results for 'CalculatorTest'. The output indicates that 2 tests passed (2 tests total, 17 ms). The test methods listed are 'testPerformIntegerDivision()' (5 ms) and 'testPerformIntegerSubtraction()' (12 ms). The console output shows the command path: '/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...' and the message: 'Process finished with exit code 0'.

Deliberately introduce a bug in the source-code



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The editor displays the 'Calculator.java' file with the following code:

```
1 package org.example.org.example;
2
3 public class Calculator { 5 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend / divisor;
7     }
8
9     public int performIntegerSubtraction(int minuend, int subtrahend) { 1 usage
10        return minuend * subtrahend;
11    }
12 }
13
```

A red error icon is visible next to line 10, indicating a compilation error due to the deliberate bug (multiplication instead of subtraction).

The 'Run' tab at the bottom shows the execution of 'CalculatorTest'. The test results indicate that 2 tests passed (2 tests total, 17 ms):

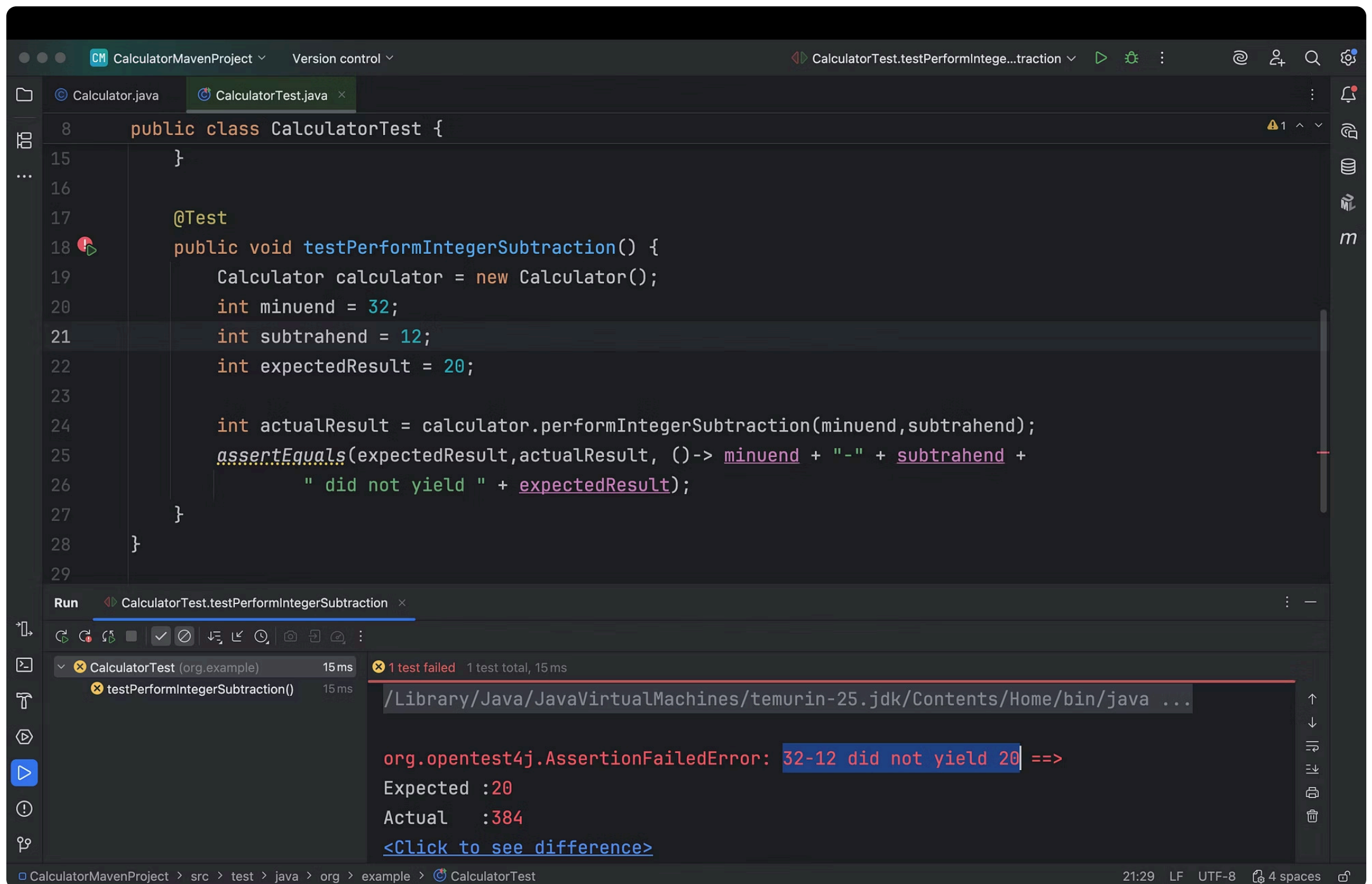
- testPerformIntegerDivision() (5 ms)
- testPerformIntegerSubtraction() (12 ms)

The output window shows the command: `/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...` and the message: `Process finished with exit code 0`.

The bottom status bar shows the current file path: `CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerSubtraction`, along with the time (10:25), encoding (LF), and file format (UTF-8).

Deliberately introduce a bug in the source-code (contd.)

Test failed!



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The editor displays the source code for 'CalculatorTest.java'. The code defines a class 'CalculatorTest' with a single test method 'testPerformIntegerSubtraction'. The test method initializes a 'Calculator' object, sets 'minuend' to 32 and 'subtrahend' to 12, and expects the result to be 20. It then calls 'performIntegerSubtraction' and asserts the result equals the expected value. A red error icon is visible next to the test method signature.

```
8 public class CalculatorTest {
15     }
16
17     @Test
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int minuend = 32;
21         int subtrahend = 12;
22         int expectedResult = 20;
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28 }
29
```

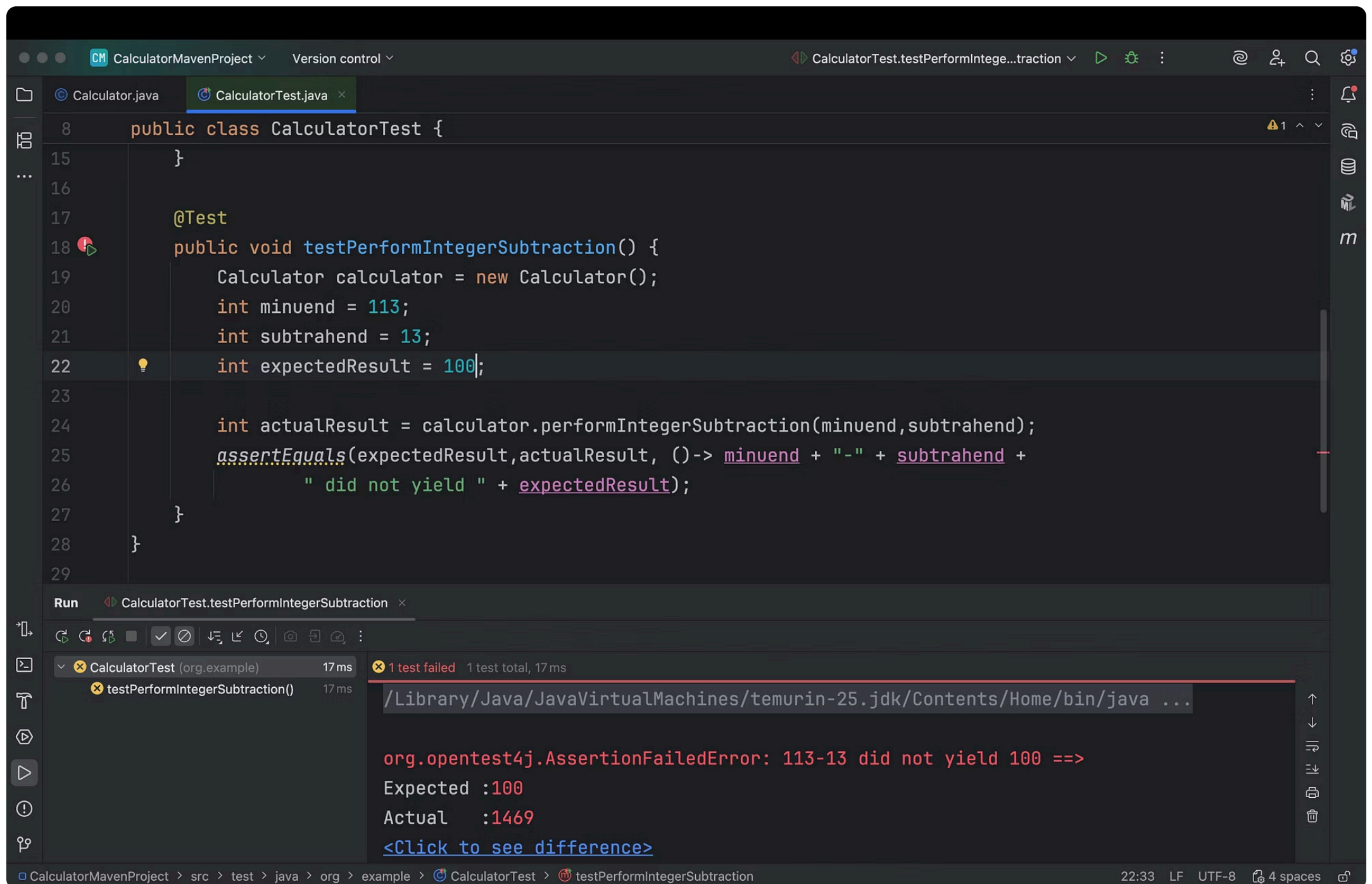
The 'Run' tab at the bottom shows the test execution results. It indicates that 1 test failed out of 1 test total, taking 15 ms. The error message is:

```
org.opentest4j.AssertionFailedError: 32-12 did not yield 20 ==>
Expected :20
Actual   :384
<Click to see difference>
```

The IDE interface includes a sidebar with file explorer, a top toolbar with various icons, and a bottom status bar showing the current file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest'.

Deliberately introduce a bug in the source-code (contd.)

Trying other input parameters. Failed!



The screenshot shows an IDE window for a project named 'CalculatorMavenProject'. The editor displays the file 'CalculatorTest.java' with the following code:

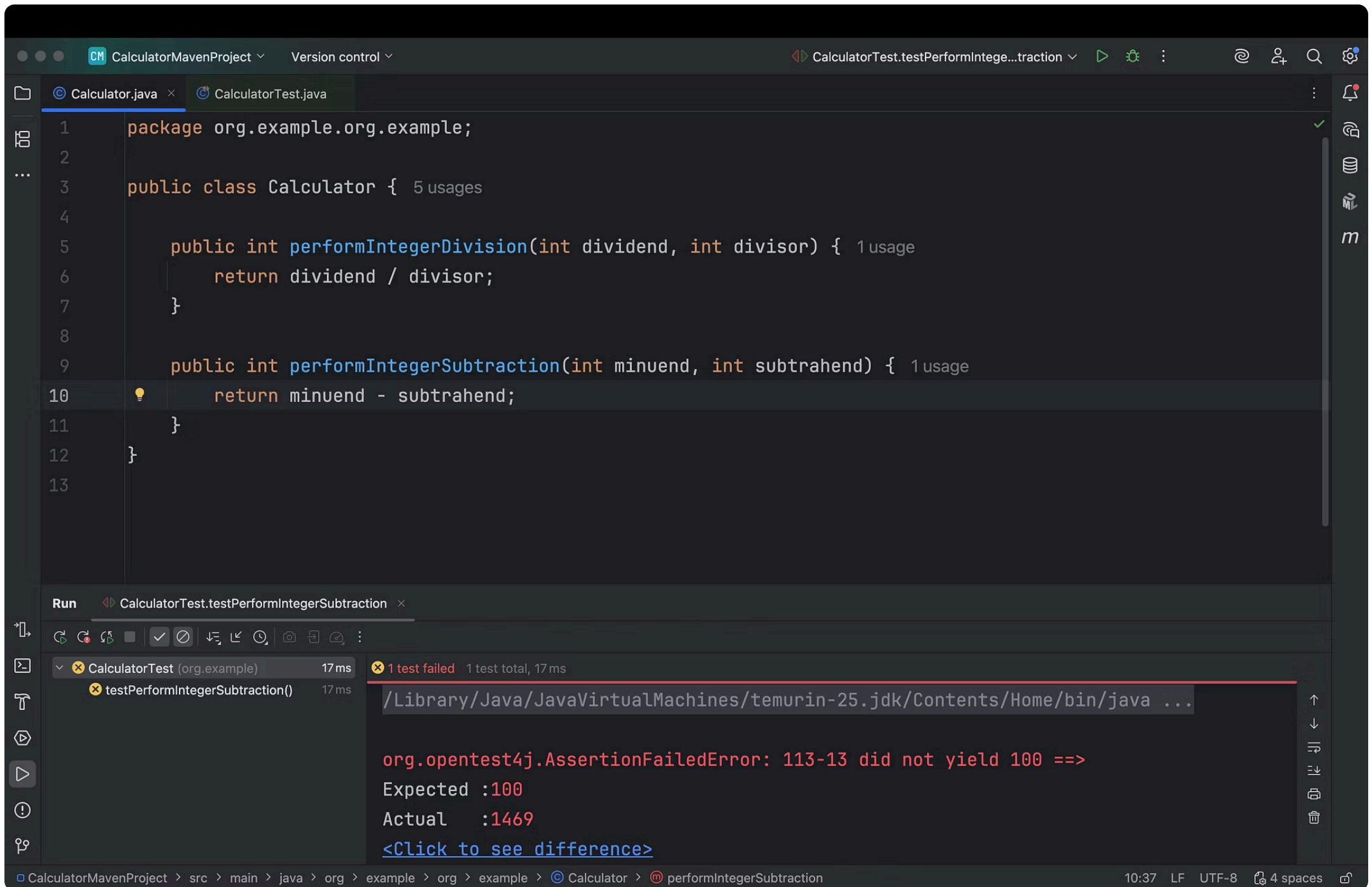
```
8 public class CalculatorTest {  
15 }  
16  
17 @Test  
18 public void testPerformIntegerSubtraction() {  
19     Calculator calculator = new Calculator();  
20     int minuend = 113;  
21     int subtrahend = 13;  
22     int expectedResult = 100;  
23  
24     int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);  
25     assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +  
26         " did not yield " + expectedResult);  
27 }  
28 }  
29
```

The 'Run' tab at the bottom shows the execution of 'CalculatorTest.testPerformIntegerSubtraction'. The output indicates that the test failed:

```
1 test failed 1 test total, 17 ms  
/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...  
org.opentest4j.AssertionFailedError: 113-13 did not yield 100 ==>  
Expected :100  
Actual   :1469  
<Click to see difference>
```

The status bar at the bottom shows the file path: 'CalculatorMavenProject > src > test > java > org > example > CalculatorTest > testPerformIntegerSubtraction'.

Fix the bug in the source code



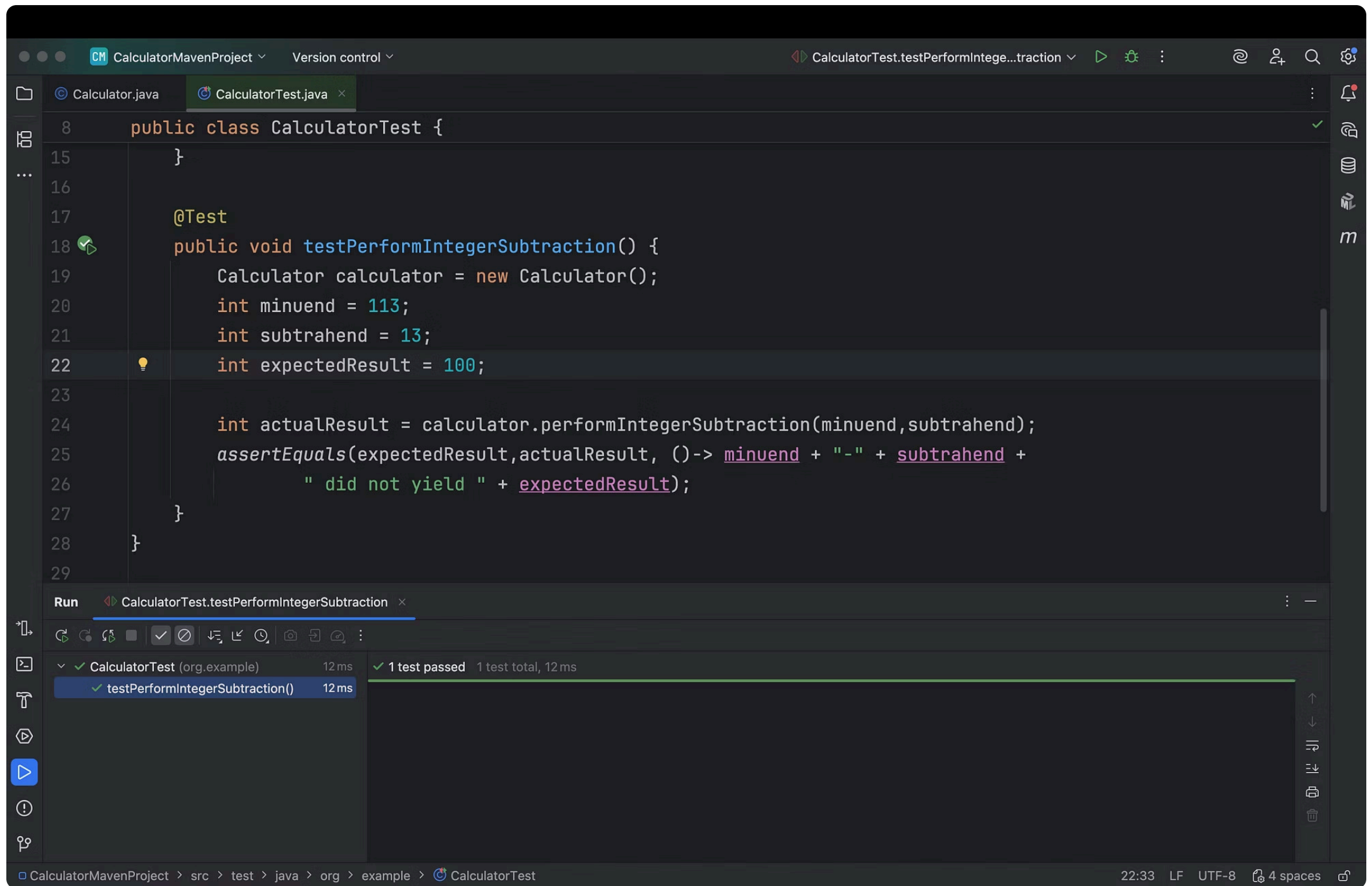
The screenshot shows an IDE window for a project named "CalculatorMavenProject". The editor displays the source code for "Calculator.java". The code defines a "Calculator" class with two methods: "performIntegerDivision" and "performIntegerSubtraction". The "performIntegerSubtraction" method contains a bug: it returns the result of "minuend - subtrahend" instead of "minuend * subtrahend". A lightbulb icon indicates a suggestion or warning at line 10.

```
1 package org.example.org.example;
2
3 public class Calculator { 5 usages
4
5     public int performIntegerDivision(int dividend, int divisor) { 1 usage
6         return dividend / divisor;
7     }
8
9     public int performIntegerSubtraction(int minuend, int subtrahend) { 1 usage
10        return minuend - subtrahend;
11    }
12 }
13
```

Below the editor, the "Run" tab shows the execution of "CalculatorTest.testPerformIntegerSubtraction". The test failed, with the message: "org.opentest4j.AssertionFailedError: 113-13 did not yield 100 ==> Expected :100 Actual :1469". The IDE also shows the path to the Java runtime: "/Library/Java/JavaVirtualMachines/temurin-25.jdk/Contents/Home/bin/java ...".

CalculatorMavenProject > src > main > java > org > example > org > example > Calculator > performIntegerSubtraction

Run the test again: Test passed!



The screenshot shows an IDE window with the following components:

- Top Bar:** Displays the project name "CalculatorMavenProject", "Version control", and the active test method "CalculatorTest.testPerformIntegerSubtraction".
- Editor:** Contains the source code for `CalculatorTest.java`. The code defines a `public class CalculatorTest` with a single test method `@Test public void testPerformIntegerSubtraction()`. Inside the method, a `Calculator` instance is created, and variables `minuend = 113`, `subtrahend = 13`, and `expectedResult = 100` are assigned. The test then calls `calculator.performIntegerSubtraction(minuend, subtrahend)` and uses `assertEquals` to verify the result. A yellow lightbulb icon indicates a suggestion for `expectedResult = 100`.
- Run Panel:** Shows the execution of the test. The command is `CalculatorTest.testPerformIntegerSubtraction`. The output table shows that the test `testPerformIntegerSubtraction()` passed in 12 ms.
- Bottom Bar:** Displays the project path `CalculatorMavenProject > src > test > java > org > example > CalculatorTest`, the time `22:33`, and encoding details `LF UTF-8 4 spaces`.

```
8 public class CalculatorTest {
15     }
16
17     @Test
18     public void testPerformIntegerSubtraction() {
19         Calculator calculator = new Calculator();
20         int minuend = 113;
21         int subtrahend = 13;
22         int expectedResult = 100;
23
24         int actualResult = calculator.performIntegerSubtraction(minuend, subtrahend);
25         assertEquals(expectedResult, actualResult, () -> minuend + "-" + subtrahend +
26             " did not yield " + expectedResult);
27     }
28 }
29
```

Test Name	Duration	Status
CalculatorTest (org.example)	12 ms	Passed
testPerformIntegerSubtraction()	12 ms	Passed

Conclusion



Enhanced Readability

Assertion messages enhance the readability and usefulness of test reports

Performance Efficiency

Lambdas help ensure performance remains efficient in large projects

Valuable Skill

Choosing when and how to include messages is a valuable unit-testing skill

Long-term Benefits

Combining clarity and efficiency strengthens long-term test maintainability

Open Floor Discussion

- Do you usually include descriptive messages in your assertions? Why or why not?
- Have you encountered slow test suites where lambda messages made a difference?
- How detailed should failure messages be to balance clarity and readability?